1. Import database / data

In [77]:
```
# import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sqlite3
```

In [78]:
```
# Data Import: db file to pandas, storing each table to a separate df

# Connect to SQLite database
conn = sqlite3.connect('customer_churn.db')

# sql query to Get all table names
sql_query = """
        SELECT name
        FROM sqlite_master
        WHERE type='table';
"""

# read sql query in pandas
tables = pd.read_sql(sql_query, conn)

# create dataframe for each table
for table_name in tables['name']:
    df = pd.read_sql(f"SELECT * FROM {table_name}", conn) # Read table into datafra
    globals()[f"df_{table_name}"] = df                    # Create dynamic datafram
    print(f"Created dataframe: df_{table_name}")

# Close connection
conn.close()
```

```
Created dataframe: df_db_customer
Created dataframe: df_db_subscription
Created dataframe: df_db_support
```

In [79]:
```
# Print table names and column names
conn = sqlite3.connect('customer_churn.db')

for table_name in tables['name']:
    print(f"\nTable Name: {table_name}")
    # Get column information
    columns_query = f"PRAGMA table_info({table_name});"   ## PRAGMA is a special co
    columns = pd.read_sql(columns_query, conn)
    print("Columns:")
    print(columns['name'].tolist())

# Close connection
conn.close()
```

```
Table Name: db_customer
Columns:
['customerid', 'name', 'country', 'state', 'gender', 'dob', 'interests', 'pincode']

Table Name: db_subscription
Columns:
['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan
_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_charge
s', 'cltv', 'churn_score']

Table Name: db_support
Columns:
['customerid', 'complaint_date', 'escalations', 'csat_score', 'col_1', 'comment']
```

2. Data cleaning

In [80]:
```
# df_db_customer.head() # customer table
df_db_customer.tail()
```

Out[80]:

	customerid	name	country	state	gender	dob	interests	pincode
16	0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00	NaN	None
17	0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00	NaN	None
18	0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00	NaN	None
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00	NaN	None
20	0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00	NaN	None

In [81]:
```
df_db_customer.info()    # to fix any missing values, nulls, which columns to drop
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 8 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   customerid  21 non-null     str
 1   name        21 non-null     str
 2   country     18 non-null     str
 3   state       21 non-null     str
 4   gender      21 non-null     str
 5   dob         21 non-null     str
 6   interests   4 non-null      str
 7   pincode     0 non-null      object
dtypes: object(1), str(7)
memory usage: 2.5+ KB
```

In [82]:
```
# a. rename col - name to customer_name
# b. drop columns - interest and pincode
# c. change data type - dob
# d. data standardization - gender   Male , Female, Men, Women
# e. fix missing values (using existing data) - country
```

a. Rename Col

In [83]:
```
# a. rename col - name to customer_name

df_db_customer.rename(columns = {'name' : 'customer_name'}, inplace= True)
```

b. Drop columns

In [84]:
```
# b. drop columns - interest and pincode                2 ways

# df_db_customer.drop(df_db_customer.columns[-2:], axis=1)     -2 se -1,        axi
# df_db_customer.drop(df_db_customer.columns[6:], axis=1)      same in positive ind

df_db_customer.drop(columns=['interests', 'pincode'], inplace=True)  # using col na
```

In [85]:
```
df_db_customer.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   customerid     21 non-null     str
 1   customer_name  21 non-null     str
 2   country        18 non-null     str
 3   state          21 non-null     str
 4   gender         21 non-null     str
 5   dob            21 non-null     str
dtypes: str(6)
memory usage: 2.2 KB
```

c. Change data type

In [86]:
```
# c. change data type - dob

df_db_customer['dob'] = pd.to_datetime(df_db_customer['dob'])       #as dob was obj


# change the rest of the columns to object one by one   if the dtype is str etc
df_db_customer['customerid'] = df_db_customer['customerid'].astype('object')
df_db_customer['customer_name'] = df_db_customer['customer_name'].astype('object')
df_db_customer['country'] = df_db_customer['country'].astype('object')
df_db_customer['state'] = df_db_customer['state'].astype('object')
df_db_customer['gender'] = df_db_customer['gender'].astype('object')
```

d. Data standardization

In [87]:
```
# d. data standardization - gender

# df_db_customer['gender'].unique()
df_db_customer['gender'] = df_db_customer['gender'].replace({'Men' : 'Male', 'Women
```

In [88]:
```
df_db_customer['gender'].unique()
```

Out[88]:
```
array(['Male', 'Female'], dtype=object)
```

e. Fix missing values

In [89]:
```
# e. fix missing values - country

# df_db_customer['country'].isna().sum()
df_db_customer[df_db_customer['country'].isna()]          #return is boolean ---> e
```

Out[89]:

	customerid	customer_name	country	state	gender	dob
5	0013-MHZWF	durga	NaN	Delhi	Female	1988-12-10
8	0015-UOCOJ	maya	NaN	Kathmandu	Female	1985-07-07
12	0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01

In [90]:
```
# country and state - unique value pair

# Creating state → country map from non-null rows;   the subset parameter is used t
state_country_mapping = df_db_customer.dropna(subset=['country']).set_index('state'

# Fill the missing country using State
df_db_customer['country'] = df_db_customer['country'].fillna(df_db_customer['state'
```

In [91]:
```
df_db_customer[df_db_customer['country'].isna()]        #To check -->  no null valu
```

Out[91]:

customerid	customer_name	country	state	gender	dob

In [92]:
```
df_db_subscription.head() # subcription table
```

Out[92]:

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	A
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	A
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	M
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	A
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	M

In [93]:
```
df_db_subscription.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   customerid              21 non-null     str
 1   subscription_start_date 21 non-null     str
 2   subscription_type       21 non-null     str
 3   renewal_date            21 non-null     str
 4   plan_type               21 non-null     str
 5   contract_type           21 non-null     str
 6   cancellation_date       6 non-null      str
 7   cancellation_reason     6 non-null      str
 8   monthly_charges         21 non-null     float64
 9   cltv                    21 non-null     int64
 10  churn_score             21 non-null     int64
dtypes: float64(1), int64(2), str(8)
memory usage: 3.1 KB
```

In [94]:
```
# change data type to date - subscription_start_date , renewal_date, cancellation_d
date_col = ['subscription_start_date', 'renewal_date', 'cancellation_date']

df_db_subscription[date_col] = df_db_subscription[date_col].apply(pd.to_datetime)
df_db_subscription.info()

# List the exact columns that are currently 'str'
str_cols = ['customerid', 'subscription_type', 'plan_type', 'contract_type', 'cance

# Convert them to object
df_db_subscription[str_cols] = df_db_subscription[str_cols].astype('object')
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   customerid              21 non-null     str
 1   subscription_start_date 21 non-null     datetime64[us]
 2   subscription_type       21 non-null     str
 3   renewal_date            21 non-null     datetime64[us]
 4   plan_type               21 non-null     str
 5   contract_type           21 non-null     str
 6   cancellation_date       6 non-null      datetime64[us]
 7   cancellation_reason     6 non-null      str
 8   monthly_charges         21 non-null     float64
 9   cltv                    21 non-null     int64
 10  churn_score             21 non-null     int64
dtypes: datetime64[us](3), float64(1), int64(2), str(5)
memory usage: 2.7 KB
```

In [95]:
```
df_db_support.head() # support table
```

Out[95]:

	customerid	complaint_date	escalations	csat_score	col_1	comment
0	0003-MKNFE	2024-08-28 00:00:00	N	60	None	service issue
1	0003-MKNFE	2024-08-28 00:00:00	Y	10	None	demaned refund
2	0013-EXCHZ	2024-01-20 00:00:00	Y	20	None	NaN
3	0013-MHZWF	2025-03-18 00:00:00	N	90	None	guidance to renew
4	0013-SMEOE	2024-11-01 00:00:00	N	30	None	NaN

In [96]:
```
df_db_support.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   customerid      9 non-null      str
 1   complaint_date  9 non-null      str
 2   escalations     9 non-null      str
 3   csat_score      9 non-null      int64
 4   col_1           0 non-null      object
 5   comment         4 non-null      str
dtypes: int64(1), object(1), str(4)
memory usage: 895.0+ bytes
```

In [97]:
```
# drop columns
df_db_support.drop(columns=['col_1', 'comment'], inplace=True)
```

In [98]:
```
# change data type to date - complaint_date

df_db_support['complaint_date'] = pd.to_datetime(df_db_support['complaint_date'])
df_db_support.info()
```

```
# List the exact string columns from this DataFrame
str_cols = ['customerid', 'complaint_date', 'escalations']

# Convert them to object
df[str_cols] = df[str_cols].astype('object')
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 4 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   customerid      9 non-null      str
 1   complaint_date  9 non-null      datetime64[us]
 2   escalations     9 non-null      str
 3   csat_score      9 non-null      int64
dtypes: datetime64[us](1), int64(1), str(2)
memory usage: 519.0 bytes
```

In [ ]:

3. Feature Engineering & Data Analysis

Create a new col

In [99]:
```
# create a new col using existing col - churn flag

# Customer is churned if cancellation_date is not null
df_db_subscription['churn_flag'] = np.where(df_db_subscription['cancellation_date']
df_db_subscription.head()
```

Out[99]:

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	A
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	A
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	M
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	A
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	M

Merge Dataframes - JOINs

In [100…
```
# first fix support table duplicates then merge  (JOIN)
# Note: while merging df's always check the shape before and after ⭐       Best pr
df = (df_db_subscription
            .merge(df_db_customer, on = 'customerid', how= 'left')        #customer
```

```
            .merge(df_db_support, on = 'customerid', how= 'left') )       # support
df.shape
```

Out[100… (23, 20)

In [101…
```
df_db_subscription.shape       # to check mismatch.    Rows and columns are mismatc
```

Out[101… (21, 12)

In [102…
```
# Effective way to check  ⭐⭐

print('df_db_subscription unique value:', df_db_subscription['customerid'].nunique(
print('df_db_customer unique value:', df_db_customer['customerid'].nunique())
print('df_db_support unique value:', df_db_support['customerid'].nunique())
print('df_db_support all value:', df_db_support['customerid'].size)
```

```
df_db_subscription unique value: 21
df_db_customer unique value: 21
df_db_support unique value: 7
df_db_support all value: 9
```

In [103…
```
df_db_support           # found duplicates
```

Out[103…

	customerid	complaint_date	escalations	csat_score
0	0003-MKNFE	2024-08-28 00:00:00	N	60
1	0003-MKNFE	2024-08-28 00:00:00	Y	10
2	0013-EXCHZ	2024-01-20 00:00:00	Y	20
3	0013-MHZWF	2025-03-18 00:00:00	N	90
4	0013-SMEOE	2024-11-01 00:00:00	N	30
5	0017-IUDMW	2024-04-10 00:00:00	Y	25
6	0019-EFAEP	2024-09-27 00:00:00	Y	30
7	0022-TCJCI	2024-09-13 00:00:00	Y	10
8	0022-TCJCI	2024-09-14 00:00:00	N	90

In [104…
```
df_db_support['complaint_count'] = df_db_support.groupby('customerid')['customerid'
```

In [105…
```
df_db_support = df_db_support.sort_values('complaint_date').drop_duplicates('custom
```

In [106…
```
df_db_support['customerid'].size
```

Out[106… 7

In [107…
```
# merge df
df = (df_db_subscription
            .merge(df_db_customer, on = 'customerid', how= 'left')
            .merge(df_db_support, on = 'customerid', how= 'left') )
```

```
df.shape
```

Out[107… (21, 21)

Export dataframe

In [108…
```
# export dataframe to csv file
df.to_csv('exported_churn_data.csv', index=False)
```

In [109…
```
 # 1. Churn Rate
```

In [110…
```
df.columns
```

Out[110…
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
       'complaint_date', 'escalations', 'csat_score', 'complaint_count'],
      dtype='str')
```

In [111…
```
churn_rate = df['churn_flag'].mean()*100
print("Churn Rate = ", round(churn_rate,2), "%")
```

```
Churn Rate =  28.57 %
```

In [112…
```
# 2. Retenion Rate
retention_rate = 100 - churn_rate
print("Retention Rate = ", round(retention_rate,2), "%")
```

```
Retention Rate =  71.43 %
```

In [113…
```
df.head(2)
```

Out[113…

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	A
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	A

2 rows × 21 columns

In [114…
```
# 3. Churn by Plan type
churn_by_plan = (df.groupby('plan_type')['churn_flag'].mean().mul(100).round(2).res
print(churn_by_plan)
```

```
  plan_type  churn_rate_pct
0     Basic           60.00
1   Premium           14.29
2  Standard           22.22
```

In [115…
```
# 5. ARPU - Avg Revenue per user
arpu = df['monthly_charges'].mean()
print('ARPU = ', round(arpu,2))
```

```
ARPU =  18.85
```

In [116…
```
# 6. Avg Customer Tenure ⭐
# count of days users has used our service : cancellation date else current date
today = pd.Timestamp.today()          # ---> func. to get todays date

df['tenure_days'] = np.where(                        # where --> for conditions   -
        df['cancellation_date'].notna(),

    (df['cancellation_date'] - df['subscription_start_date']).dt.days,

    (today - df['subscription_start_date']).dt.days
)

avg_tenure = df['tenure_days'].mean()
print("Avg Tenure (Days) = ", round(avg_tenure),0)
```

```
Avg Tenure (Days) =  1564 0
```

In [117…
```
# 7. Revenue at risk - revenue lost from churned users
revenue_at_risk = df.loc[df['churn_flag']==1, 'monthly_charges'].sum()     # 1 mean
print("Revenue at Risk (Rs 'K') =", revenue_at_risk)
```

```
Revenue at Risk (Rs 'K') = 73.94
```

In [118…
```
# 8. Esclation Rate
escalation_rate = (df['escalations']=='Y').mean()*100                  # escalatio
print("Esclation Rate = ", round(escalation_rate, 2), "%")
```

```
Esclation Rate =  19.05 %
```

In [119…
```
# 9. Avg Complaint Per User
avg_complaints = df['complaint_count'].sum() / df['customerid'].nunique()   # divid
print("Avg Compliants Per User = ", round(avg_complaints, 2))
```

```
Avg Compliants Per User =  0.43
```

In [120…
```
# 10. Correlation Esclation vs Churn  ⭐⭐

df['escalations'] = np.where(df['escalations'] == 'Y', 1, 0) # encoding string to i
corr_df = df[['escalations', 'churn_flag']].dropna()            # use double sq bra

correlation = corr_df['escalations'].corr(df['churn_flag'])                    # c
print("Correlation between esclation vs churn is = ", round(correlation,2))
```

```
Correlation between esclation vs churn is =  0.77
```

In [121…
```
# 11. Create a column using existing col - Churn risk     ⭐⭐    conditions passe
conditions = [
        (df['churn_score'] < 50),
        (df['churn_score'] >= 50) & (df['churn_score'] < 70),
        (df['churn_score'] >= 70)
]
```

```
choices = ['low', 'med', 'high']

df['churn_risk'] = np.select(conditions, choices, default='unkown')     # new colum
```

In [122…
```
df[['churn_risk', 'churn_score']].tail()
```

Out[122…

	churn_risk	churn_score
16	med	62
17	low	27
18	high	99
19	low	7
20	low	47

4. Visualization using Matplotlib

In [123…
```
# best practice to create a copy then work on it
df_visual = df.copy()        # method
```

In [124…
```
df_visual.columns          # show all columns
```

Out[124…
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
       'complaint_date', 'escalations', 'csat_score', 'complaint_count',
       'tenure_days', 'churn_risk'],
      dtype='str')
```

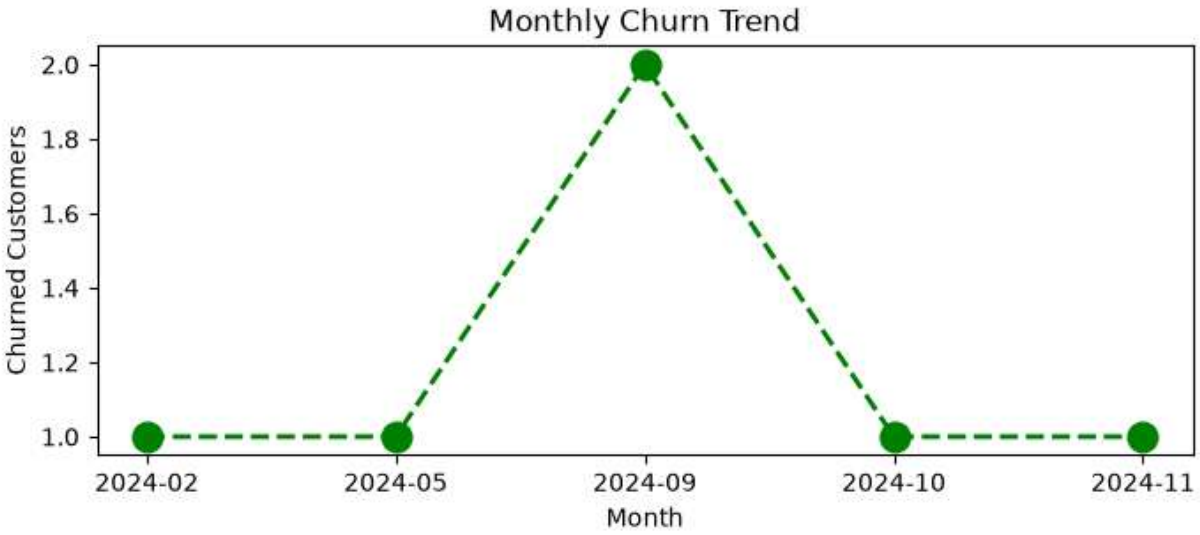
In [125…
```
# 4.1 Monthly Churn Trend (Time Series KPI)

df_visual['cancellation_month'] = df_visual['cancellation_date'].dt.to_period('M')

churn_trend = df_visual[df_visual['churn_flag'] == 1].groupby('cancellation_month')

plt.figure(figsize=(8,3))
plt.plot(churn_trend.index.astype(str), churn_trend.values,     # x axis (index) as
         color='green', marker='o', linestyle='dashed',  linewidth=2, markersize=12

plt.title('Monthly Churn Trend')
plt.xlabel('Month')
plt.ylabel('Churned Customers')
plt.show()
```



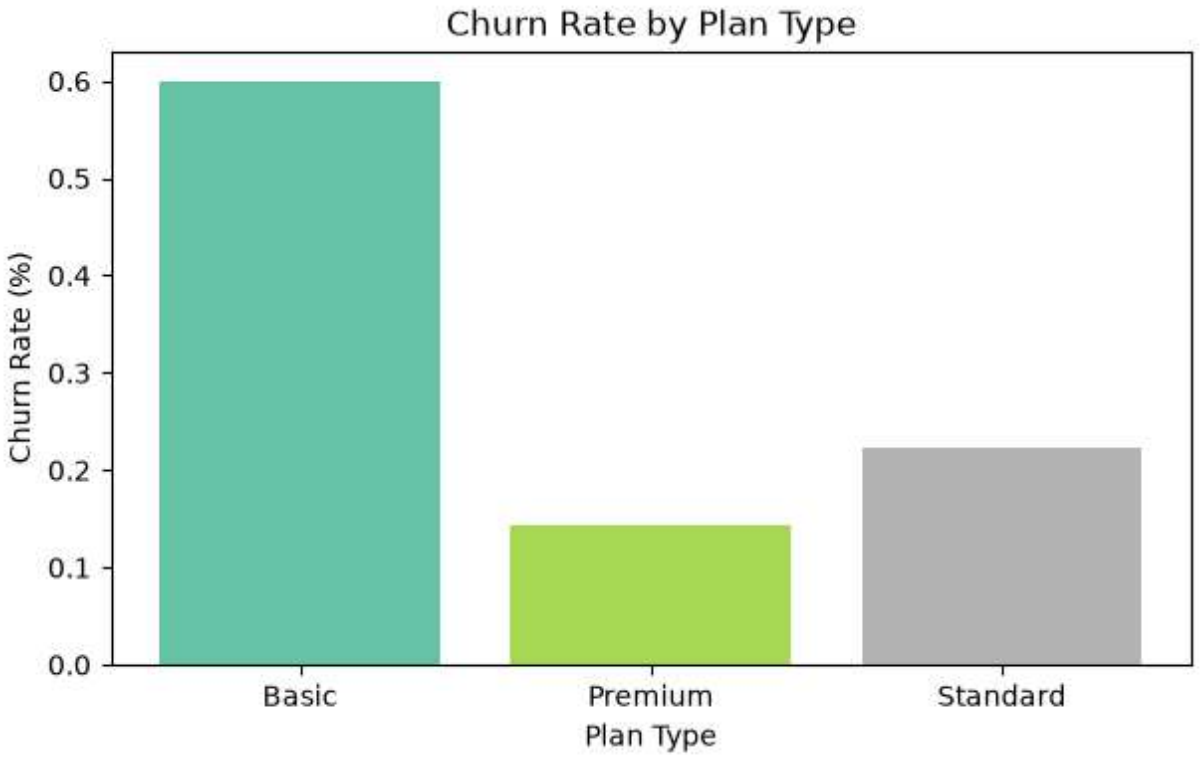
In [126…

```
# 4.2 Churn Rate by Plan type
churn_plan = df_visual.groupby('plan_type')['churn_flag'].mean()

# colors = ['yellow', 'purple', 'blue']
colors = plt.cm.Set2(np.linspace(0, 1, len(churn_plan)))

plt.figure(figsize=(7,4))
plt.bar(churn_plan.index, churn_plan.values, color = colors)

plt.title('Churn Rate by Plan Type')
plt.xlabel('Plan Type')
plt.ylabel('Churn Rate (%)')
plt.show()
```

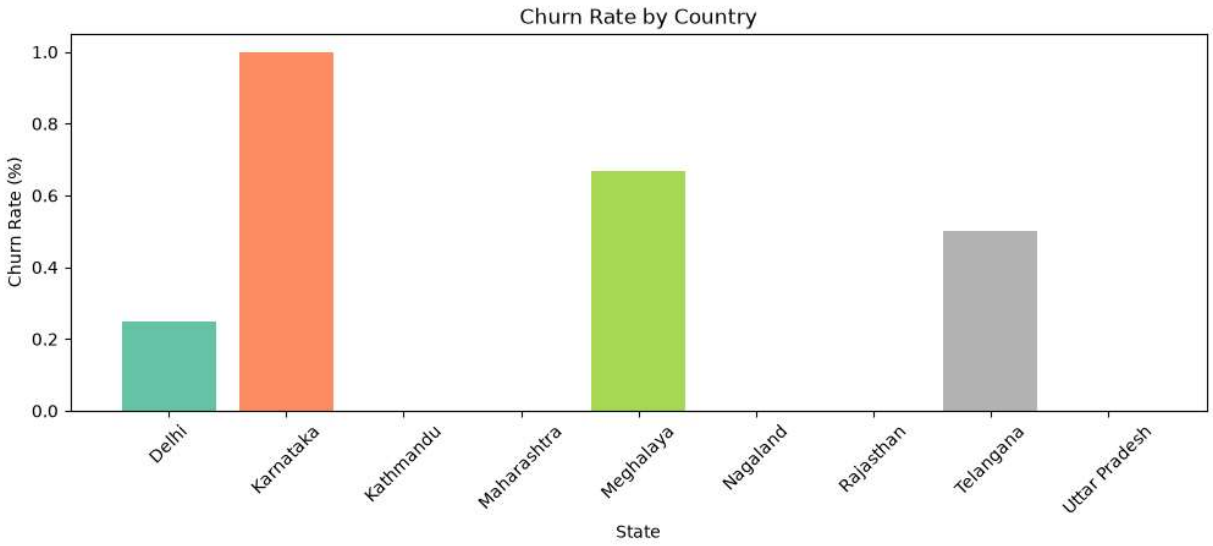


In [127…
```
# 4.3 Churn by States
churn_plan = df_visual.groupby('state')['churn_flag'].mean()

# colors = ['yellow', 'purple', 'blue']
colors = plt.cm.Set2(np.linspace(0, 1, len(churn_plan)))

plt.figure(figsize=(12,4))
plt.bar(churn_plan.index, churn_plan.values, color = colors)

plt.title('Churn Rate by Country')
plt.xlabel('State')
plt.ylabel('Churn Rate (%)')
plt.xticks(rotation=45)
plt.show()
```



In [ ]:

Visulaization using Seaborn

In [128…
```
# Heatmap (Correlation Matrix)
```

In [129…
```
# encoding - convert str to numeric so that we can find corr between features in he
df_visual.columns
```

Out[129…
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
       'complaint_date', 'escalations', 'csat_score', 'complaint_count',
       'tenure_days', 'churn_risk', 'cancellation_month'],
      dtype='str')
```

In [130…
```
df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag', 'churn_risk',
```

Out[130…

	plan_type	contract_type	churn_score	churn_flag	churn_risk	escalations
0	Standard	Annual	12	0	low	0
1	Premium	Annual	91	1	high	1
2	Basic	Monthly	34	0	low	0
3	Premium	Annual	8	0	low	0
4	Standard	Monthly	88	1	high	1

In [131…

```
# remove warnings
import warnings
warnings.filterwarnings("ignore")
```

In [132…

```
# incorrect method of encoding - as numbers are not assigned based on priority ⭐⭐
df_encoded = df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag',

categorial_cols = ['plan_type', 'contract_type', 'churn_risk']

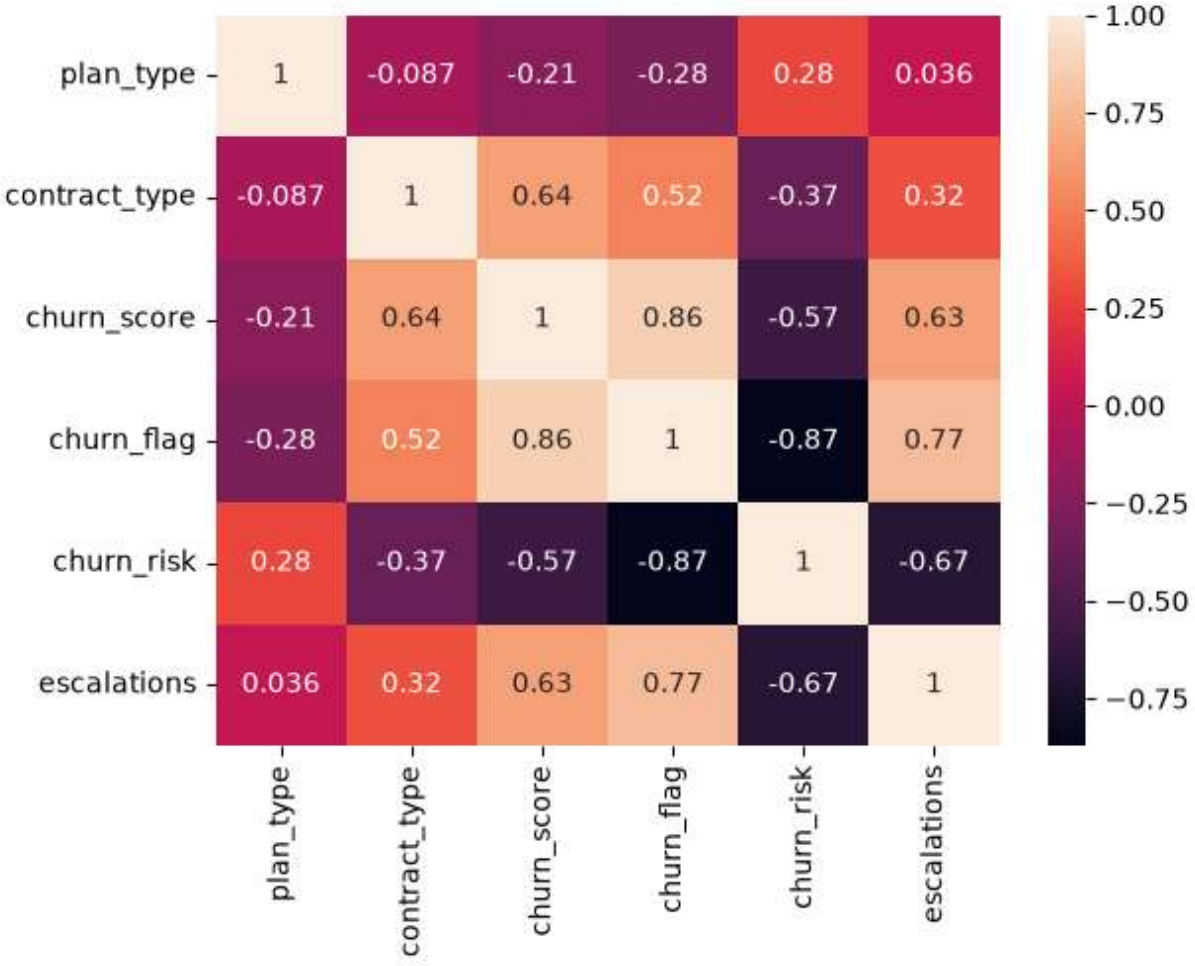
for col in categorial_cols:
    df_encoded[col] = df_encoded[col].astype('category').cat.codes
```

In [133…

```
# Heatmap (correlation matrix)

sns.heatmap(df_encoded.corr(), annot=True)
```

Out[133… <Axes: >



In [134…
```
print('plan_type :', df_visual['plan_type'].unique())
print('contract_type :', df_visual['contract_type'].unique())
print('churn_risk :', df_visual['churn_risk'].unique())
```

```
plan_type : ['Standard' 'Premium' 'Basic']
contract_type : ['Annual' 'Monthly']
churn_risk : <ArrowStringArray>
['low', 'high', 'med']
Length: 3, dtype: str
```

In [135…
```
# Correct method of encoding - based on priority
df_encoded = df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag',

order_mappings = {
    'plan_type' : ['Basic', 'Standard', 'Premium'],
    'contract_type' : ['Monthly', 'Annual'],
    'churn_risk': ['low', 'med', 'high']
    }

for col, order in order_mappings.items():
    df_encoded[col] = pd.Categorical(df_encoded[col].astype('category'), categories
```

In [136…
```
df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag', 'churn_risk',
```

Out[136…

	plan_type	contract_type	churn_score	churn_flag	churn_risk	escalations
0	Standard	Annual	12	0	low	0
1	Premium	Annual	91	1	high	1
2	Basic	Monthly	34	0	low	0
3	Premium	Annual	8	0	low	0
4	Standard	Monthly	88	1	high	1

In [137…

```
df_encoded.head()     #  converts words into numbers by grouping them and giving ea
```

Out[137…

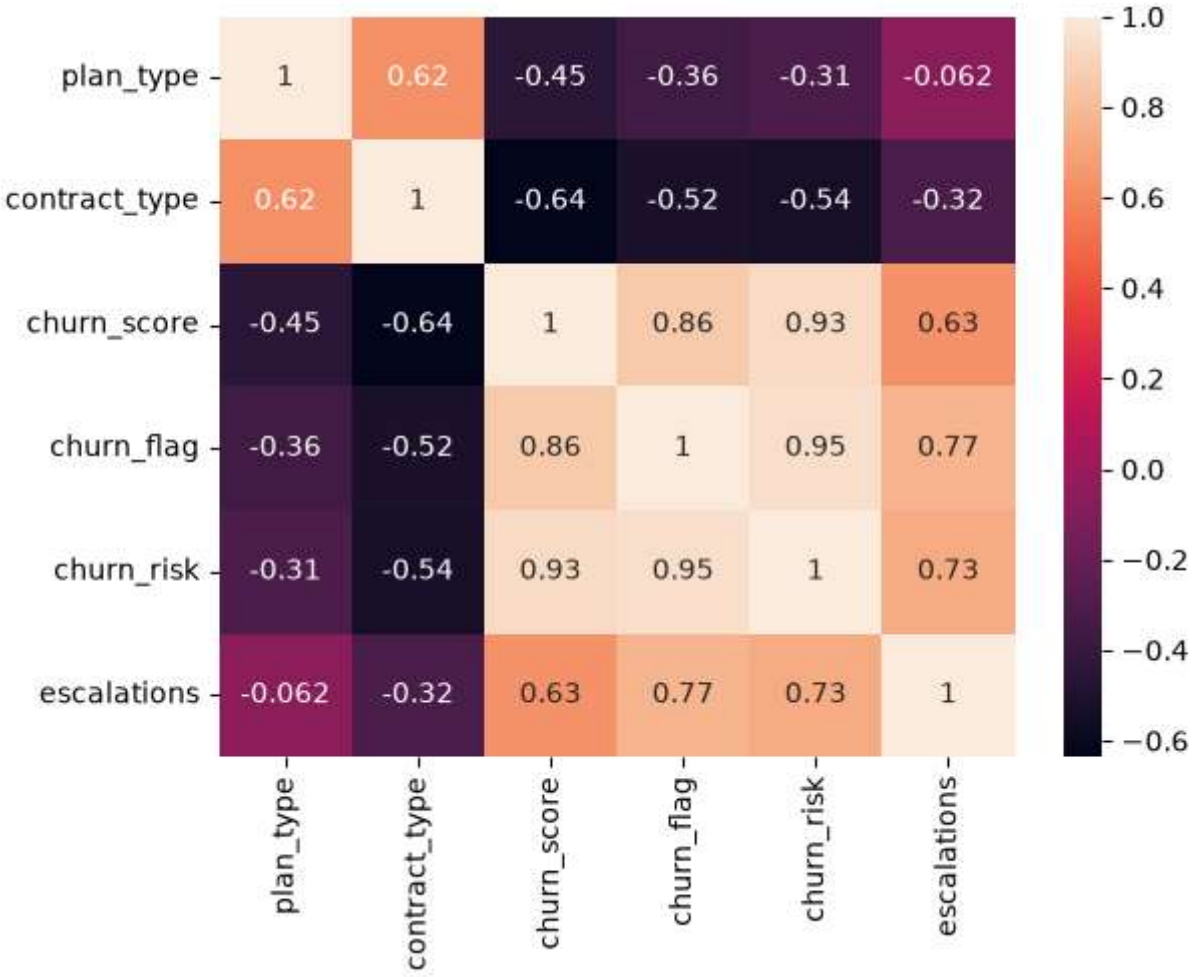
	plan_type	contract_type	churn_score	churn_flag	churn_risk	escalations
0	1	1	12	0	0	0
1	2	1	91	1	2	1
2	0	0	34	0	0	0
3	2	1	8	0	0	0
4	1	0	88	1	2	1

In [138…

```
# Heatmap (correlation matrix)
sns.heatmap(df_encoded.corr(), annot=True)
```

Out[138…

```
<Axes: >
```



In [ ]:

Pivot table

In [139…

```
# Pivot Table

pd.pivot_table(
    df_visual,
    index='plan_type',
    values='churn_flag',
    aggfunc = 'mean'
)
```

Out[139…

plan_type	churn_flag
Basic	0.600000
Premium	0.142857
Standard	0.222222

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: